

山东大学



网络安全创新创业实践

secp256k1 分析

姓名：杨阳 学号：202300460010

姓名：孙琬竺 学号：202300460020

姓名：胡子玮 学号：202300460032

姓名：熊芷萱 学号：202300460163

2026 年 7 月 22 日

secp256k1 分析

目录

1	背景与前置准备	1
1.1	项目要求	1
1.2	背景	1
2	secp256k1 曲线的特殊性与自同态	2
3	自同态等价定理	2
3.1	$\varphi(P)$ 在曲线上	3
3.2	φ 与 λ	3
3.3	β 与 λ	3
4	GLV 标量分解	3
4.1	目标	3
4.2	格视角	3
4.3	最近向量和 Babai 舍入	4
4.4	用移位乘法代替除法	4
4.5	误差界的严格证明	4
4.6	Eisenstein 整数视角	5
5	性能优化	5
5.1	最新 signed-digit 重构	5
6	Bug 修复	5
6.1	统一加法公式	6
6.2	端态触发原因	6
6.3	修复	6
6.4	两个表达式等价	6
7	commit 历史	7
8	代码复现数学结论	7
9	思考	8
10	附录	9
10.1	附录 A: 团队重点阅读源码位置	9
10.2	附录 B: 参考文献	9
10.3	附录 C: 分工	10

1 背景与前置准备

1.1 项目要求

项目要求：研究 bitcoin-core/secp256k1 仓库，就其中一处与”密码学算法在比特币中的使用”相关的 bug 修复 / 性能优化写一份报告，弄清楚为什么以及背后的数学。

关注官方专门写了设计文档或 sage 数学验证脚本的模块,sage 中有 `gen_split_lambda_constants.sage`, 是生成 GLV 分解常数 $g1/g2/b1/b2$ 的脚本, 证明它们不是硬编码魔数, 而是从数学推导来的; `weierstrass_prover.sage` / `group_prover.sage` / `prove_group_implementations.sage` 三个用符号计算证明点加/倍点公式实现正确; `secp256k1_params.sage` 是曲线参数 p, n, a, b, G ; `gen_exhaustive_groups.sage` 是生成小阶测试群。

secp256k1 / sage /

Justsomebuddy sage: verify Eisenstein integer connection for GLV constants		25
Name	Last commit message	
..		
gen_exhaustive_groups.sage	Introduce SECP256K1_8 macro for curve b coefficient	
gen_split_lambda_constants.sage	sage: verify Eisenstein integer connection for GLV constants	
group_prover.sage	small fixes	
prove_group_implementations.sage	Merge #1078: group: Save a normalize_to_zero in gej_add_ge	
secp256k1_params.sage	Fix G.y parity in sage code	
weierstrass_prover.sage	sage: Ensure that constraints are always fastfracs	

查看到 commit 5de4c5d 那个 bug, 作者在代码注释里写了:

```
1 - If y1 = -y2 but x1 != x2, which does occur thanks to certain
2   properties of our curve (specifically, 1 has nontrivial cube
3   roots in our field, and the curve equation has no x coefficient)
4   then the answer is not infinity but also not given by the above equation.
5
6 /* This only occurs when y1 == -y2 and x1^3 == x2^3, but x1 != x2.
7    * This means either x1 == beta*x2 or beta*x1 == x2, where beta is
8    * a nontrivial cube root of one. */
```

可以看出曲线存在非平凡立方根正好就是性能优化能成立的同一个数学性质, 即同一条数学性质, 一手给了性能, 一手埋了 bug。

本报告选取的主题是 GLV 端态优化, 它是 secp256k1 相对通用椭圆曲线库快近一倍的核心原因, 同时:

- 是一处性能优化, 标量乘法 doubling 步数减半
- 引入了一处必须修复的退化 bug, `gej_add_ge` 的 0/0 情形, 详情为 commit 5de4c5d
- 背后是一套完整而优雅的数学, 包括单位立方根、格约减 / Eisenstein 整数、Babai 舍入误差界证明

1.2 背景

比特币的每一笔交易输入都要用 ECDSA 签名/验签。课程幻灯片里的 ECDSA 为:

- 签名 $Sign(m)$: 随机 k , 计算 $R = kG$, $r = R_x \pmod{n}$, $s = k^{-1}(e + dr) \pmod{n}$ 。
- 验签 $Verify(r, s)$: 计算 $w = s^{-1}$, $R' = (e \cdot w)G + (r \cdot w)P$, 检查 $R'_x == r$ 。

无论签名还是验签, 最昂贵的运算都是椭圆曲线标量乘法 $k \cdot P$, 把一个点 P 加上自己 k 次, k 是一个 256-bit 的整数, 用二进制展开需要约 256 次倍点加上平均约 128 次点加。

一个比特币全节点在初始区块同步时要验证从创世块至今的每一笔历史交易签名, 数量在数十亿级别, 标量乘法的常数因子哪怕只快 10%, 落到全网所有节点上都是巨大的算力节约, 这就是 secp256k1 库做底层优化的动机。

2 secp256k1 曲线的特殊性与自同态

比特币采用的曲线 secp256k1 定义在素域 F_p 上:

$$p = 2^{256} - 2^{32} - 977$$

$$E: y^2 = x^3 + 7, a = 0, b = 7$$

n = 曲线阶

端态优化能成立的前提:

1. $a = 0$ (方程里没有一次项 x), 曲线的 j -不变量为 0;
2. $p \equiv 1 \pmod{3}$, 因此 F_p 中存在非平凡的单位立方根 β ($\beta^3 = 1, \beta \neq 1$); 同理曲线阶 $n \equiv 1 \pmod{3}$, Z_n 中存在非平凡单位立方根 λ 。

有了这两点, 就能定义一个高效可计算的自同态:

$$\varphi: E \rightarrow E, \varphi(x, y) = (\beta \cdot x, y)$$

注意只把 x 坐标乘上常数 β , y 保持不变; 成本仅一次域乘法, 几乎无成本。

源码中这两个常数 (`src/field.h` 与 `src/scalar_impl.h`):

```
1 /* : F_p 上的本原立方根 */
2 static const secp256k1_fe secp256k1_const_beta = SECP256K1_FE_CONST(
3     0x7ae96a2bul, 0x657c0710ul, 0x6e64479eul, 0xac3434e9ul,
4     0x9cf04975ul, 0x12f58995ul, 0xc1396c28ul, 0x719501eeul
5 );
6
7 /* : Z_n 上的本原立方根 */
8 static const secp256k1_scalar secp256k1_const_lambda = SECP256K1_SCALAR_CONST(
9     0x5363AD4CUL, 0xC05C30E0UL, 0xA5261C02UL, 0x8812645AUL,
10    0x122E22EAUL, 0x20816678UL, 0xDF02967CUL, 0x1B23BD72UL
11 );
```

3 自同态等价定理

为什么 $\varphi(P) = \lambda P$, 是整个优化成立的关键定理。

3.1 $\varphi(P)$ 在曲线上

设 $P = (x, y) \in E$, 即 $y^2 = x^3 + 7$ 。代入 $\varphi(P) = (\beta x, y)$:

$$(\beta x)^3 + 7 = \beta^3 \cdot x^3 + 7 = 1 \cdot x^3 + 7 = x^3 + 7 = y^2$$

关键一步用到了 $\beta^3 = 1$ 以及方程里没有 x 一次项, 否则 $\beta \cdot x$ 项无法消去。这也解释为什么 $a = 0$ 是必要条件。所以 $\varphi(P)$ 满足曲线方程, φ 是 $E \rightarrow E$ 的映射。

3.2 φ 与 λ

φ 不仅是映射, 还是群同态。在一个阶为素数 n 的循环群上, 任何自同态都等价于乘以某个固定整数。设这个整数为 λ , 即对所有 P 有:

$$\varphi(P) = \lambda \cdot P$$

再看 φ 作用三次 $\varphi^3(x, y) = (\beta^3 x, y) = (x, y)$, 即 $\varphi^3 = \text{id}$ 。于是 $\lambda^3 \equiv 1 \pmod{n}$, 且 $\lambda \neq 1$ (因为 φ 不是恒等), 所以 λ 是 Z_n 上的本原立方根。

3.3 β 与 λ

由 $X^3 - 1 = (X - 1)(X^2 + X + 1)$, 本原立方根是 $X^2 + X + 1 = 0$ 的根, 因此:

$$\beta^2 + \beta + 1 \equiv 0 \pmod{p} \quad \lambda^2 + \lambda + 1 \equiv 0 \pmod{n}$$

等价地 $\beta^2 \equiv -\beta - 1$, $\lambda^2 \equiv -\lambda - 1$ 。

4 GLV 标量分解

4.1 目标

GLV 方法的思路是算 $k \cdot P$, 先把大标量 k 拆成两个半长标量:

$$k \equiv k_1 + \lambda \cdot k_2 \pmod{n}, \text{ 其中 } |k_1|, |k_2| \leq \sqrt{n} \approx 2^{128}$$

$$\text{那么 } k \cdot P = k_1 \cdot P + k_2 \cdot (\lambda P) = k_1 \cdot P + k_2 \cdot \varphi(P)$$

$\varphi(P)$ 几乎免费, 于是问题从一个 256-bit 标量乘法变成两个 128-bit 标量乘法。用 Straus/Shamir 技巧把两组同时进行, 倍点次数从约 256 降到约 128。

4.2 格视角

问题是如何找到小的 (k_1, k_2) , 我们考虑所有把 k 映为 0 的分解构成的格:

$$L = \{(u, v) \in Z^2 : u + \lambda \cdot v \equiv 0 \pmod{n}\}$$

这是 Z^2 的一个子格, 行列式等于 n 。由 Minkowski 定理, 它存在一组约化基, 两个基向量长度都在 $\sqrt{n} \approx 2^{128}$ 量级。源码注释中给出了这组约化基 (a_1, b_1) 与 (a_2, b_2) , 均为 128-bit 量级:

```
1 a1 = 0x3086d221a7d46bcde86c90e49284eb15
2 b1 = -0xe4437ed6010e88286f547fa90abfe4c3
3 a2 = 0x0114ca50f7a8e2f3f657c1108d9d44cfd8
4 b2 = 0x3086d221a7d46bcde86c90e49284eb15 (= a1)
```

并且满足 $a_1 + b_1 \cdot \lambda \equiv 0$ 和 $a_2 + b_2 \cdot \lambda \equiv 0 \pmod{n}$, 以及 $d = a_1 \cdot b_2 - b_1 \cdot a_2 = n$ 。

4.3 最近向量和 Babai 舍入

给定 k , 向量 $(k, 0)$ 显然满足 $k + \lambda \cdot 0 = k$ 。我们在格 L 里找一个最接近 $(k, 0)$ 的格点, 相减得到的余量就是那个短的 (k_1, k_2) 。用 Babai 最近平面求解, 把 $(k, 0)$ 用约化基展开为实数系数, 四舍五入到整数:

```
1 c1 = round( b2 * k / n )
2 c2 = round( -b1 * k / n )
3 k2 = -(c1 * b1 + c2 * b2)
4 k1 = k - lambda * k2
```

四舍五入引入的误差 $\leq \pm 1/2$ 个基向量, 因此 k_1, k_2 的界被约化基长度 $\approx 2^{128}$ 控制住。

4.4 用移位乘法代替除法

除法在计算机计算中代价十分高昂。secp256k1 把除以 n 换成乘以一个预计算的高精度倒数再右移。预计算常数 (src/scalar_impl.h):

```
1 g1 = round(2^384 * b2 / d)
2 g2 = round(2^384 * (-b1) / d)
```

于是 $c_1 = \text{round}(k \cdot g_1 / 2^{384})$ 、 $c_2 = \text{round}(k \cdot g_2 / 2^{384})$, 即一次 mul_shift。对应代码:

```
1 /* these _var calls are constant time since the shift amount is constant */
2 secp256k1_scalar_mul_shift_var(&c1, k, &g1, 384); /* c1 = round(k*g1 / 2^384) */
3 secp256k1_scalar_mul_shift_var(&c2, k, &g2, 384); /* c2 = round(k*g2 / 2^384) */
4 secp256k1_scalar_mul(&c1, &c1, &minus_b1);
5 secp256k1_scalar_mul(&c2, &c2, &minus_b2);
6 secp256k1_scalar_add(r2, &c1, &c2); /* r2 = k2 */
7 secp256k1_scalar_mul(r1, r2, &secp256k1_const_lambda);
8 secp256k1_scalar_negate(r1, r1);
9 secp256k1_scalar_add(r1, r1, k); /* r1 = k - *k2 */
```

4.5 误差界的严格证明

源码里附了一段严谨证明, 核心是把总误差拆成两部分:

$$\varepsilon_i = 2^{256} \cdot |g_i / 2^{384}|$$

$$|c_1 - k \cdot b_2 / n| \leq |c_1 - k \cdot g_1 / 2^{384}| + |k \cdot g_1 / 2^{384} - k \cdot b_2 / n| \leq 1/2 + 2^{256} \cdot |g_1 / 2^{384} - b_2 / d| = 1/2 + \varepsilon_1$$

把这些误差代回 k_1, k_2 的表达式, 最终证明:

```
1 或者 r1 < 2^128, 或者 (-r1 mod n) < 2^128
2 或者 r2 < 2^128, 或者 (-r2 mod n) < 2^128
```

即两个分量的绝对值都 ≤ 128 bit (含符号)。这个界是后续 WNAF of lambda_split output has max size 129 (出现在 commit 2edc514) 等优化能安全成立的前提。

4.6 Eisenstein 整数视角

约化基并非“凑出来”的，它源于一个漂亮的代数结构：设 $\omega = (-1 + \sqrt{-3})/2$ ($X^2 + X + 1 = 0$ 的复根)，则 $Z[\omega]$ 是 Eisenstein 整数环，一个欧几里得整环。环同态 $\varphi: Z[\omega] \rightarrow Z_n, \varphi(a + b\omega) = a + b\lambda$ 的核恰好就是上面的格 L 。由于 $Z[\omega]$ 有良好的除法，约化基可由 n 在 $Z[\omega]$ 中的分解直接得到。仓库新增了 sage 脚本 `sage/gen_split_lambda_constants.sage` 生成并用 `sage: verify Eisenstein integer connection` (PR # 1800) 验证这些常数确实来自 Eisenstein 整数结构，而非硬编码的魔数。

5 性能优化

拆分后， $k \cdot P = k_1 \cdot P + k_2 \cdot (\lambda P)$ ，两个标量都只有约 128 bit。计算 λP 只需对预计算表里每个点的 x 坐标乘 β ：

```
1 secp256k1_fe_mul(&r->x, &r->x, &secp256k1_const_beta); /* 由 A 表快速得到 A 表 */
```

然后用 Straus / 交错方法同时扫描 $k_1 k_2$ 的比特：

	朴素 double-and-add	GLV 端态
标量长度	256 bit	2×128 bit (并行)
倍点 doubling 次数	~ 256	~ 128
点加 addition 次数	~ 128 (wNAF 更少)	两组各 $\sim$ 约一半
额外成本	—	一次 <code>split_lambda</code> + 每个表点一次域乘

倍点次数减半是主要加速来源。

5.1 最新 signed-digit 重构

早期实现用 fixed-wNAF + skew 修正来保证奇数倍查表。commit 40f50d0 用带符号数字算法重写了 `ecmult_const`，思路更纯粹：

- 令 $s = f(q)$ ，用 `split_lambda` 得 $s = s_1 + \lambda \cdot s_2$ ；
- 取 $v_1 = s_1 + 2^{128} v_2 = s_2 + 2^{128}$ (保证为正、 n bit 长)；
- 把结果写成 $q \cdot A = \sum (2 \cdot v_1[i] - 1) \cdot 2^i \cdot A + \sum (2 \cdot v_2[i] - 1) \cdot 2^i \cdot (\lambda A)$
其中 $x[i]$ 是 x 的第 i 位——即根据比特把 $\pm 2^i \cdot A$ 累加起来；
- 只要选 $f(q) = (q + (1 + \lambda)(2^n - 2^{129} - 1))/2 \pmod{n}$ ，结果就精确等于 $q \cdot A$ 。

好处是不再需要末尾的 skew 修正，全部用标量运算决定查表索引，代码更简单、更快，可以看到作者实测约 +2%，且预计算表大小可独立调优再得小幅加速。这一步同时新增了 `secp256k1_scalar_half` (标量减半) 等基础函数。

6 Bug 修复

端态优化在带来性能的同时，也制造了一个隐蔽的正确性 bug。

6.1 统一加法公式

为了常数时间（抗侧信道，见幻灯片提到的“侧信道泄露私钥”威胁），secp256k1 的点加 `gej_add_ge` 用了一个对两点相加和倍点都成立的统一公式。公式内部要算一个辅助斜率 $\lambda_{add} = R/M$ ，这里的 λ_{add} 是加法公式里的中间量，与端态的立方根 λ 无关。其中 $M = Y_1 \cdot Z_2^3 + Y_2 \cdot Z_1^3$ 一类的量，即雅可比坐标下的 $y_1 + y_2$ 。

- 当 $P = -Q$ （即 $x_1 = x_2, y_1 = -y_2$ ）：结果是无穷远点 0，代码单独置 `infinity` 标志处理；
- 但存在一个更隐蔽的情形： $y_1 = -y_2$ 却 $x_1 \neq x_2$ 。此时 $M = 0$ 且分子 $R = 0$ ，公式算出 $\lambda_{add} = 0/0$ ，结果彻底错误。

6.2 端态触发原因

$y_1 = -y_2$ 意味着 $y_1^2 = y_2^2$ ，代入曲线方程 $y^2 = x^3 + 7$ ： $x_1^3 + 7 = x_2^3 + 7 \implies x_1^3 = x_2^3$

若同时 $x_1 \neq x_2$ ，则 $(x_1/x_2)^3 = 1$ 且 $x_1/x_2 \neq 1$ —— x_1/x_2 恰好是一个非平凡立方根 β ！即 $x_1 = \beta \cdot x_2$ 或 $\beta \cdot x_1 = x_2$

这个退化情形正是一个点与另一个点通过端态 β 相关时出现的。而端态优化的整个运算过程，计算 $k_1 \cdot P + k_2 \cdot \varphi(P)$ ，其中 $\varphi(P) = (\beta x, y)$ 会大量地把这类 x 坐标差一个 β 因子的点喂进 `gej_add_ge`。于是这个平时几乎不可能被随机数据触发的退化点，在端态优化下变成了必然会遇到的输入。

6.3 修复

`commit 5de4c5d` 的做法是：当检测到 $0/0$ 时，用另一个数学上等价、但在此处有定义的斜率表达式替换： $\lambda_{add} = (y_1 - y_2)/(x_1 - x_2)$

关键代码 (`src/group_impl.h`):

```

1  /* 检测退化: M 和 R 同时归零 */
2  degenerate = secp256k1_fe_normalizes_to_zero(&m) &
3              secp256k1_fe_normalizes_to_zero(&rr);
4
5  /* 备用分子/分母: rr_alt = Y1*Z2^3 - Y2*Z1^3, m_alt = X1*Z2^2 - X2*Z1^2 */
6  secp256k1_fe_negate(&rr_alt, &s2, 1);
7  secp256k1_fe_add(&rr_alt, &s1);
8  secp256k1_fe_negate(&m_alt, &u2, 1);
9  secp256k1_fe_add(&m_alt, &u1);
10
11 /* 非退化时用原式，退化时用备用式——用 cmov 做到常数时间、无分支 */
12 secp256k1_fe_cmov(&rr_alt, &rr, !degenerate);
13 secp256k1_fe_cmov(&m_alt, &m, !degenerate);

```

6.4 两个表达式等价

在两者都有定义的地方， R/M 与 $(y_1 - y_2)/(x_1 - x_2)$ 相等：把前者上下同乘 $(y_1 + y_2)/(y_1 + y_2)$ ，再用曲线方程 $y^2 = x^3 + 7$ 把 y^2 替换成 $x^3 + 7$ 消元，即可互相推出。而且可以证明：对任意一对非零点，两个表达式中至少有一个是有定义的，因此二者用 `cmov` 组合就覆盖了所有情况。

且必须用 `cmov` 而不是 `if`, 如果写成 `if (degenerate) { ... }`, 分支会依赖点的坐标——而点的坐标由私钥 `k` 决定。攻击者可通过时序 / 侧信道观察分支是否命中, 从而推断私钥比特。这正是 ECDSA 攻击侧信道泄露私钥的现实版本, `secp256k1_fe_cmov` 无论条件真假都执行相同的内存/算术操作, 只是有条件地选择结果, 从而做到常数时间。

7 commit 历史

经过我们查阅资料与阅读历史 `commit`, 发现 GLV 端态优化并非一直默认开启:

- GLV 方法曾被 Certicom 的相关专利覆盖, 早期 `secp256k1` 用一个编译开关 `USE_ENDOMORPHISM` 把它设为可选, 默认关闭, 以规避法律风险;
- 相关专利在 2020 年前后到期;
- 随后仓库在 `commit 4232e5b Rip out non-endomorphism code` 与 `commit c6b6b8f Rip out non-endomorphism code + dependencies` 中删除了非端态代码路径, 端态优化成为唯一且默认的实现。

8 代码复现数学结论

团队书写了纯 Python、零依赖的验证脚本 `verify_glv.py`, 把报告中的每一条数学结论都亲手跑一遍。所有常数直接摘自 `secp256k1` 源码, 脚本自行实现了 F_p 上的椭圆曲线运算与 `split_lambda` 分解。运行结果完整输出见 `verify_output.txt`:

- 验证 3 实测每个分量都落在 125–128 bit, 从数据上印证了第 4 节 256-bit 标量降为两个 ~ 128 -bit 标量的界; 同时验证了点乘等式 $k \cdot G = k_1 \cdot G + k_2 \cdot \varphi(G)$, 即端态优化在数值上确实成立。
- 验证 4 用一对真实构造的点 $P_1 = (x, y)$ 、 $P_2 = (\beta x, -y)$ 直接命中第 6 节的退化情形: 统一公式分母 $M = y_1 + y_2 = 0$ ($0/0$), 而备用斜率 $(y_1 - y_2)/(x_1 - x_2)$ 算出的结果与标准点加完全一致, 且答案是有限点 $(1 - \lambda) \cdot Q$ 而非无穷远。

运行 `python verify_glv.py`:

```
> python verify_glv.py
```

```
=====
验证 1:  $\beta$ 、 $\lambda$  是非平凡单位立方根
=====
```

```
[通过]  $\beta^3 \equiv 1 \pmod{p}$ 
[通过]  $\beta \neq 1$ 
[通过]  $\beta^2 + \beta + 1 \equiv 0 \pmod{p}$ 
[通过]  $\lambda^3 \equiv 1 \pmod{n}$ 
[通过]  $\lambda \neq 1$ 
[通过]  $\lambda^2 + \lambda + 1 \equiv 0 \pmod{n}$ 
```

```
=====
验证 2: 自同态  $\phi(x,y)=(\beta x,y)$  等于 乘以  $\lambda$ , 即  $\phi(P) = \lambda \cdot P$ 
=====
```

```
[通过] G 在曲线上
[通过]  $\phi(G)$  在曲线上
[通过]  $\phi(G) == \lambda \cdot G$ 
[通过] 随机点 P0:  $(\beta x,y) == \lambda \cdot P$ 
[通过] 随机点 P1:  $(\beta x,y) == \lambda \cdot P$ 
[通过] 随机点 P2:  $(\beta x,y) == \lambda \cdot P$ 
```

```
=====
验证 3: GLV 分解  $k = k_1 + \lambda \cdot k_2 \pmod{n}$ , 且  $k_1, k_2 \sim 2^{128}$ 
=====
```

```
k(256bit) 拆成 ->  $|k_1|=126\text{bit}, |k_2|=126\text{bit}$ , 且  $k_1+\lambda k_2 == k$  ✓
k(256bit) 拆成 ->  $|k_1|=125\text{bit}, |k_2|=126\text{bit}$ , 且  $k_1+\lambda k_2 == k$  ✓
k(256bit) 拆成 ->  $|k_1|=127\text{bit}, |k_2|=126\text{bit}$ , 且  $k_1+\lambda k_2 == k$  ✓
k(256bit) 拆成 ->  $|k_1|=126\text{bit}, |k_2|=127\text{bit}$ , 且  $k_1+\lambda k_2 == k$  ✓
k(256bit) 拆成 ->  $|k_1|=128\text{bit}, |k_2|=126\text{bit}$ , 且  $k_1+\lambda k_2 == k$  ✓
[通过] 所有分量位长  $\leq 129$  bit (原始 256 bit 减半)
[通过]  $k \cdot G == k_1 \cdot G + k_2 \cdot \phi(G)$  (GLV 点乘等式)
```

```
=====
验证 4: 退化 bug 根源 —  $x_1=\beta \cdot x_2$  且  $y_1=-y_2$  触发统一公式 0/0
=====
```

```
[通过] P1, P2 均在曲线上
[通过]  $x_1 \neq x_2$ 
[通过]  $x_1^3 == x_2^3$  (立方相等)
[通过]  $x_1 == \beta^2 \cdot x_2$  (差一个非平凡立方根)
[通过]  $y_1 == -y_2$  (触发  $M=y_1+y_2=0$ )
统一公式分母  $M = y_1+y_2 = 0$  -> 若不修复即 0/0
[通过] 统一公式分母  $M == 0$  (bug 触发点)
[通过] 备用斜率分母  $(x_1-x_2) \neq 0$  (修复可用)
[通过] 备用斜率  $(y_1-y_2)/(x_1-x_2)$  算出的和 == 标准点加结果
[通过]  $P_1+P_2 == (1-\lambda) \cdot Q$  (有限点, 非无穷远)
```

9 思考

- GLV 把标量乘法的 doubling 从 ~ 256 降到 ~ 128 , 这是历史上引入端态时约 20%–30% 整体点乘加速的主要来源;
- signed-digit 重构在此基础上再约 +2%, 并显著简化代码、消除 skew 修正;
- `gej_add_ge` 的退化修复用 `cmov` 保证了在使用端态、且要求恒定时间的场景 (如 ECDH、Schnorr 签名 nonce 相关运算) 下, 既正确又不泄露私钥。

全节点在 IBD 阶段要验证数十亿历史签名, 矿池节点在高峰期每秒要做大量验签。标量乘法快近一倍意味着同步更快、验证吞吐更高、硬件成本更低。secp256k1 也因此成为业界公认最快的

ECC 库之一，被大量非比特币项目复用。

GLV 端态优化是一个把数论巧合，secp256k1 特意选取 $a = 0$ 、 $p \equiv 1 \pmod{3}$ 使 F_p 、 Z_n 都有非平凡立方根转化为工程性能的典范：

1. 数学上，自同态 $\varphi(x, y) = (\beta x, y)$ 等于“乘以立方根 λ ”，几乎免费地提供 λP ；标量分解 $k = k_1 + \lambda k_2$ 通过格约减整数结构 + Babai 舍入把 256-bit 标量降为两个 128-bit 标量，并有严格的误差界证明。
2. 倍点次数减半，历史上带来约 20–30% 点乘加速，后续 signed-digit 重构再优化。
3. Bug 修复方面，同一个立方根存在的数学性质，反过来让统一加法公式 `gej_add_ge` 出现 0/0 退化 ($x_1 = \beta x_2$ 且 $y_1 = -y_2$)；commit 5de4c5d 用等价备用斜率 $(y_1 - y_2)/(x_1 - x_2)$ 配合常数时间 `cmov` 修复，兼顾正确性与抗侧信道。

10 附录

10.1 附录 A：团队重点阅读源码位置

内容	文件	关键符号 / commit
曲线参数 $p, n, a=0, b=7$	<code>src/scalar_impl.h</code> , <code>sage/secp256k1_params.sage</code>	—
立方根 β (域)	<code>src/field.h:69</code>	<code>secp256k1_const_beta = 0x7ae96a2b...719501ee</code>
立方根 λ (标量)	<code>src/scalar_impl.h:83</code>	<code>secp256k1_const_lambda = 0x5363AD4C...1B23BD72</code>
标量分解	<code>src/scalar_impl.h</code>	<code>secp256k1_scalar_split_lambda</code> , 常数 <code>minus_b1/minus_b2/g1/g2</code>
分解误差界证明	<code>src/scalar_impl.h</code> (注释)	<code>split_lambda</code> 上方 + <code>split_lambda_verify</code>
常数生成 / 溯源	<code>sage/gen_split_lambda_constants.sage</code>	PR #852, #1800 (Eisenstein 验证)
端态点乘	<code>src/ecmult_const_impl.h</code> , <code>src/ecmult_impl.h:316</code>	signed-digit 算法, PR #1184
Bug 修复	<code>src/group_impl.h (gej_add_ge)</code>	commit 5de4c5d, <code>degenerate / m_alt / rr_alt / cmov</code>
专利后清理	全库	commit 4232e5b, c6b6b8f

10.2 附录 B：参考文献

- R. Gallant, R. Lambert, S. Vanstone, “Faster Point Multiplication on Elliptic Curves with Efficient Endomorphisms”, CRYPTO 2001.
- D. Hankerson, A. Menezes, S. Vanstone, “Guide to Elliptic Curve Cryptography”, 算法 3.74 与 § 3.5 (源码注释直接引用)。
- É. Brier, M. Joye, “Weierstraß Elliptic Curves and Side-Channel Attacks”, PKC 2002 (统一加法公式)。
- Gouvêa, Oliveira, López, “Efficient Software Implementation of Public-Key Cryptography on Sensor Networks Using the MSP430X”, § 4.3 (g_1/g_2 高精度估计)。
- `bitcoin-core/secp256k1` 仓库源码与提交历史 (本报告所有 commit 哈希均取自该仓库)。

10.3 附录 C：分工

姓名（学号）	具体分工内容
杨阳（202300460010）	- 研读比特币里标量乘法为什么是性能瓶颈 - 研读 secp256k1 曲线参数”特殊性 ($a = 0$、$p \equiv 1 \pmod{3}$、非平凡立方根) - 证明 $\varphi(x, y) = (\beta x, y)$ 还在曲线上、$\varphi(P) = \lambda P$、β/λ 是本原立方根、$\beta^2 + \beta + 1 \equiv 0$
孙琬竺（202300460020）	- 研读 split_lambda 的目标 $k = k_1 + \lambda k_2$, 分量降到 $\sim 2^{128}$ - 研读格约减视角核格 L、约化基 (a1,b1)(a2,b2)、行列式 $=n$ - Babai 舍入和用 g1/g2 移位代替除法的工程技巧 - 误差界证明
胡子玮（202300460032）	- ecmult_const 如何用端态加速、signed-digit 重构 - 研读重点 bug:gej_add_ge 的 0/0 退化情形 - 书写 bug 根源 ($x_1 = \beta x_2$ 且 $y_1 = -y_2$)、cmov 修复、为什么要常数时间 - 串联 bug 和性能
熊芷萱（202300460163）	- commit 历史探究 - 探索性能数据、对比特币 IBD 的意义、结论 - 跑 verify_glv.py 验证脚本 - 整合文档